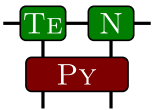
Matrix product states for physics: Tensor Network Python (TeNPy)

Johannes Hauschild

Technical University Munich

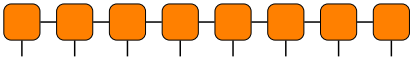
24.03.2025

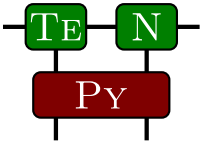




https://github.com/tenpy/tenpy_toycodes

Outline





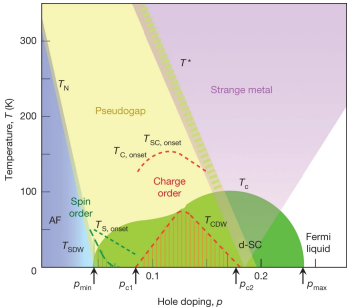
Intro to MPS, TEBD, DMRG

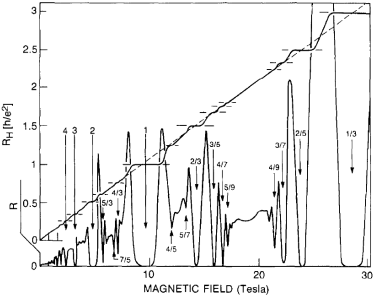
1. Motivation
2. Exercise 1: Basics
3. MPS
   - MPS Expectation Values
   - MPO for Hamiltonian
4. Exercise 2: MPS and Model
5. TEBD time evolution
6. DMRG ground state search
7. Exercise 3: DMRG

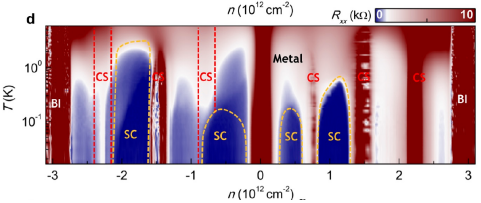
Intro to TeNPy

8. Overview
9. Models
10. Simulations
11. Exercise 4: TeNPy

Historical Motivation: condensed matter systems are quantum many body







Nature 518, 179 (2015), Physica B 177 (1992), Nature 574, 653 (2019)

Classical Simulations of Quantum Many Body Systems

Goal

numerically solve

$$\hat{H}\,|\psi_0\rangle = E_0\,|\psi_0\rangle$$
$$i\hbar\partial_t\,|\psi(t)\rangle = \hat{H}\,|\psi(t)\rangle$$

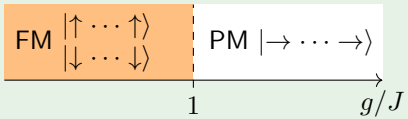
on classical computers

$$|\psi\rangle \in \mathcal{H} = \bigotimes_{i=0}^{N} \mathbb{C}^d$$

Example (Transverse Field Ising Model)

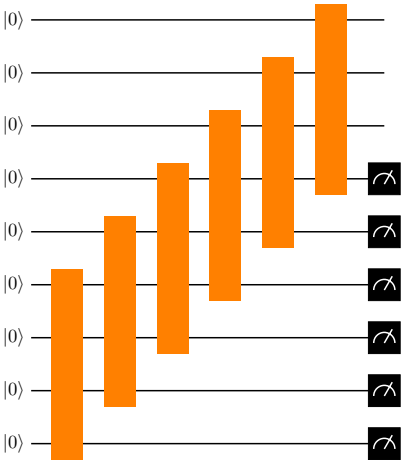
$$\hat{H} = -J\sum_{i=1}^{N-1}\hat{Z}_i\hat{Z}_{i+1} - g\sum_{i=1}^{N}\hat{X}_i$$

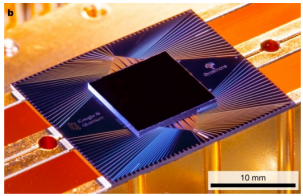
$$\hat{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \hat{Z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$



$\Rightarrow$ challenge: exponentially big $\dim\mathcal{H} = d^N$

Modern Motivation: Quantum Computing



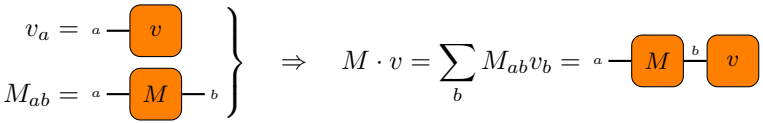


Google, Nature 574, 505 (2019)

Use tensor networks to

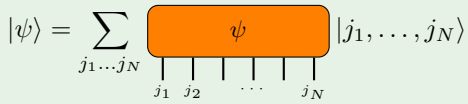
- provide benchmark data
- simulate (some) cirucits
- raise the bar for quantum advantage
- inspire quantum algorithms

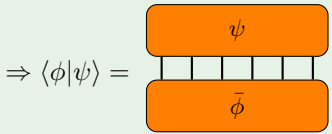
Graphical Notation



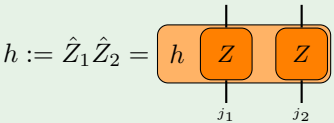
$\Rightarrow$ familiar example: bulk of quantum circuits

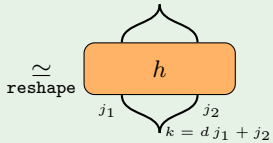
Example (overlap)



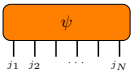


Example (reshape)



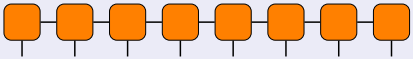


Overview: Tensor Networks

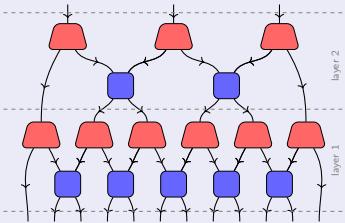
general idea: variational ansatz $|\psi\rangle = \sum_{j_1 \ldots j_N}$ 

$|j_1, \ldots, j_N\rangle$ with ψ tensor network

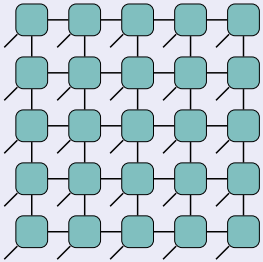
MPS (here)



MERA

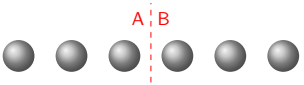


PEPS



and infinite versions
and more ...

Schmidt decomposition



$$|\psi\rangle = \sum_{k_A,k_B} \psi \; |k_A\rangle\,|k_B\rangle$$

$$\overset{\text{SVD}}{=} \sum_{k_A,k_B} A \;\alpha\; \Lambda \;\alpha'\; B \; |k_A\rangle\,|k_B\rangle$$

$$= |\alpha\rangle_A \; A \;\alpha\; \Lambda \;\alpha'\; B \; |\alpha'\rangle_B$$

with diagonal $\Lambda_{\alpha,\alpha'} = \underbrace{\lambda_\alpha}_{\geq 0}\,\delta_{\alpha,\alpha'}$

SVD Singular Value Decomposition

matrix decomposition

$$M = USV^\dagger$$

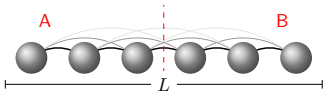
with diagonal $S \geq 0$ and isometries

$$U^\dagger U = \mathbb{1} \qquad V^\dagger V = \mathbb{1}$$

$${}_A\langle\bar{\alpha}|\alpha\rangle_A = \; A \;\alpha\; \bar{A} \;\bar{\alpha}\; = \; \alpha \; \bar{\alpha} \; = \delta_{\alpha,\bar{\alpha}}$$

$$\delta_{\alpha',\bar{\alpha}'} = \; \alpha' \; \bar{\alpha}' \; = \; \alpha' \; B \; \bar{\alpha}' \; \bar{B} \; = {}_B\langle\bar{\alpha}'|\alpha'\rangle_B$$

Area Law for Entanglement Entropy



Entanglement entropy

$$S = -\operatorname{Tr} \varrho_A \log(\varrho_A) = -\sum_\alpha \lambda_\alpha^2 \log \lambda_\alpha^2$$

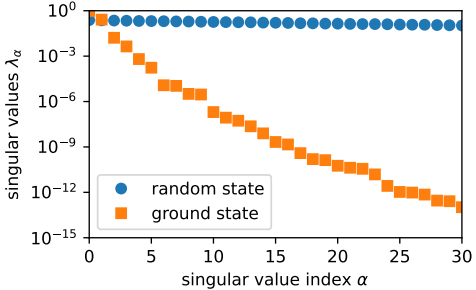
where $\varrho_A = \operatorname{Tr}_B |\psi\rangle \langle\psi| = A \Lambda^2 A^\dagger$

Area law

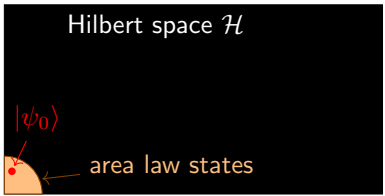
for ground states of gapped Hamiltonians

$S \propto$ area of cut $= \mathcal{O}(L^{D-1}) \underset{\text{in 1D}}{=} \text{const in } L$

Hastings (J.Stat.Mech. 2007)

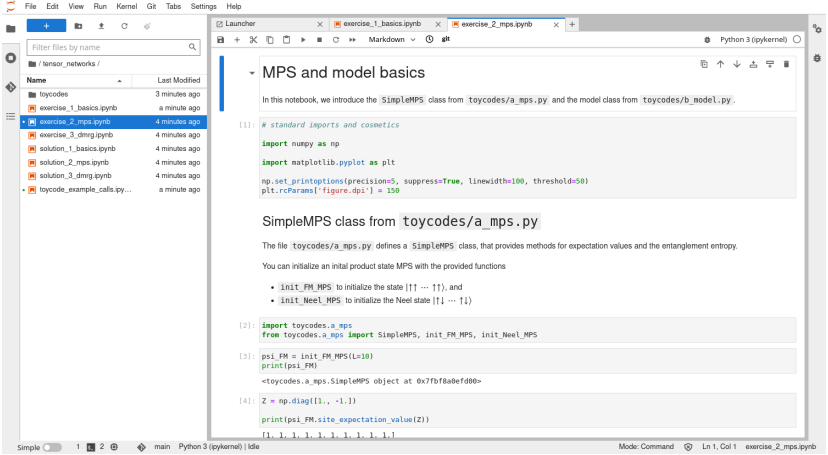


$\Rightarrow$ can truncate: only keep χ largest λ_α



Exercise 1: Basics

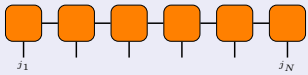
https://github.com/tenpy/tenpy_toycodes



Matrix Product States (MPS)

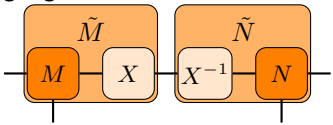
Definition (MPS)

Variational Ansatz

$$|\psi\rangle = \sum_{j_1 \ldots j_N}$$  $$|j_1, \ldots, j_N\rangle$$

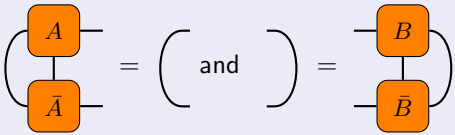
How to initialize given $|\psi\rangle$?

gauge freedom on contracted legs:

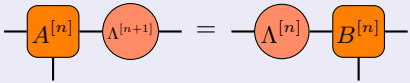


Canonical Form

ONB of Schmidt states



in canonical form with

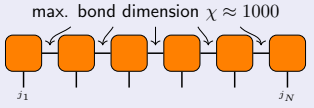


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

$$|\psi\rangle = \sum_{j_1 \dots j_N}$$

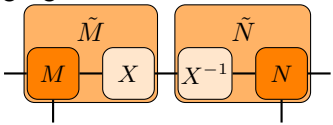


$$|j_1, \dots, j_N\rangle$$

free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

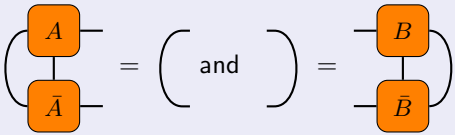
How to initialize given $|\psi\rangle$?

gauge freedom on contracted legs:

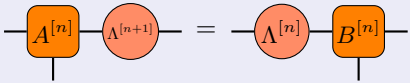


Canonical Form

ONB of Schmidt states



in canonical form with

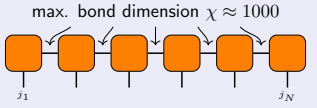


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

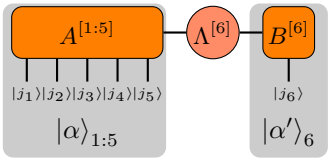
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



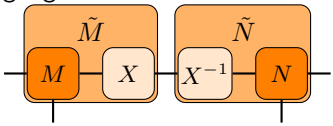
$$|j_1, \dots, j_N\rangle$$

free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?

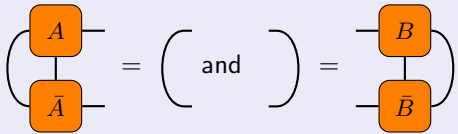


gauge freedom on contracted legs:

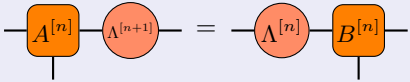


Canonical Form

ONB of Schmidt states



in canonical form with

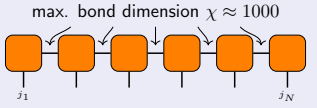


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

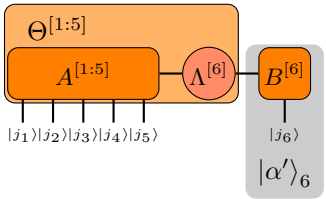
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



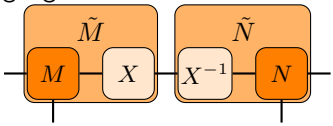
$$|j_1, \dots, j_N\rangle$$

free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?

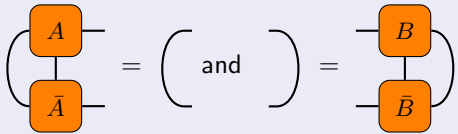


gauge freedom on contracted legs:

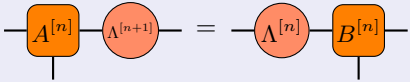


Canonical Form

ONB of Schmidt states



in canonical form with

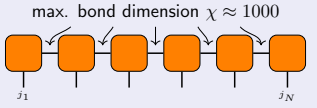


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

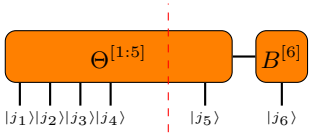
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



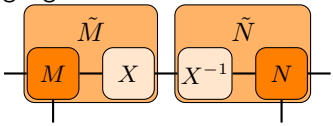
$$|j_1, \dots, j_N\rangle$$

free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?

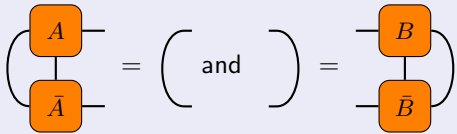


gauge freedom on contracted legs:

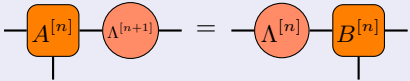


Canonical Form

ONB of Schmidt states



in canonical form with

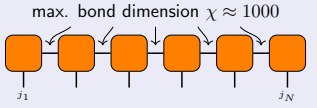


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

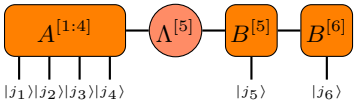
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



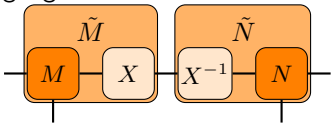
$$|j_1, \dots, j_N\rangle$$

free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?

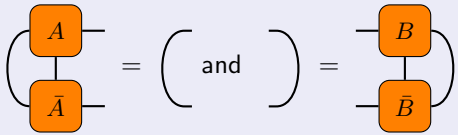


gauge freedom on contracted legs:

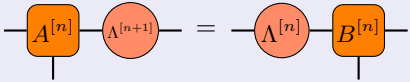


Canonical Form

ONB of Schmidt states



in canonical form with

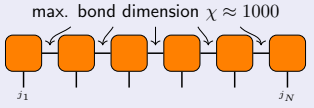


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

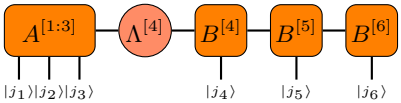
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



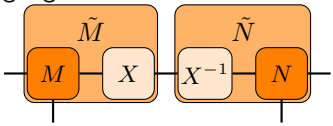
$$|j_1, \dots, j_N\rangle$$

free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?

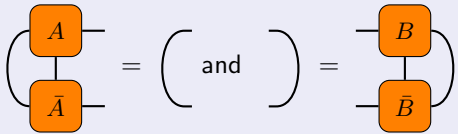


gauge freedom on contracted legs:

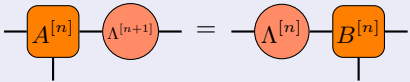


Canonical Form

ONB of Schmidt states



in canonical form with

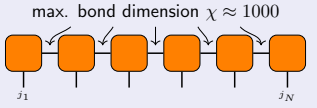


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

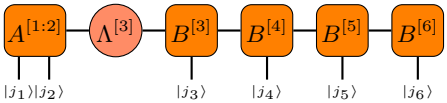
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



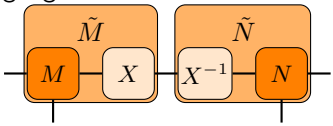
$$|j_1, \dots, j_N\rangle$$

free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?

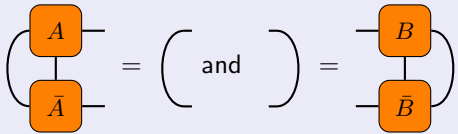


gauge freedom on contracted legs:

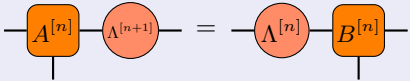


Canonical Form

ONB of Schmidt states



in canonical form with

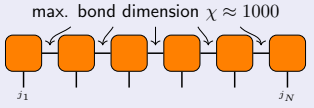


Matrix Product States (MPS)

Definition (MPS)

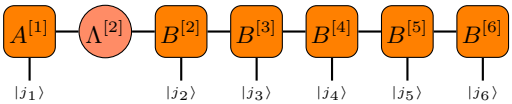
Variational Ansatz

$$|\psi\rangle = \sum_{j_1 \dots j_N} \quad |j_1, \dots, j_N\rangle$$

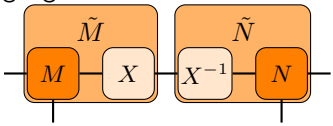


free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?

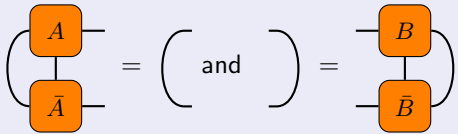


gauge freedom on contracted legs:

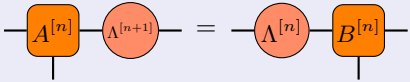


Canonical Form

ONB of Schmidt states



in canonical form with

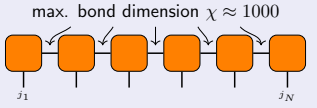


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

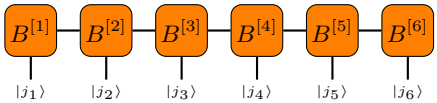
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



$$|j_1, \dots, j_N\rangle$$

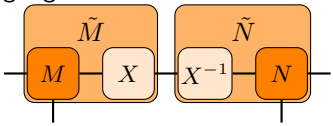
free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?



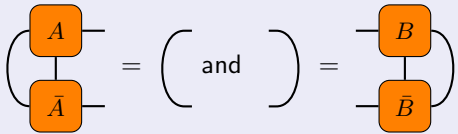
$\to$ in *right* canonical form

gauge freedom on contracted legs:

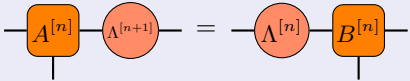


Canonical Form

ONB of Schmidt states



in canonical form with

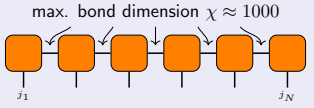


Matrix Product States (MPS)

Definition (MPS)

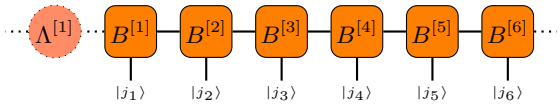
Variational Ansatz

$$|\psi\rangle = \sum_{j_1 \dots j_N} \quad |j_1, \dots, j_N\rangle$$



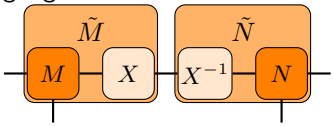
free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?



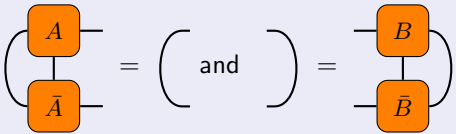
$\to$ in *right* canonical form

gauge freedom on contracted legs:

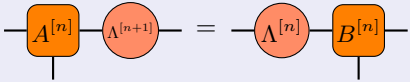


Canonical Form

ONB of Schmidt states



in canonical form with

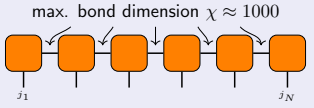


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

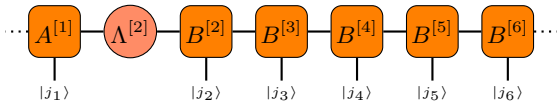
$$|\psi\rangle = \sum_{j_1 \cdots j_N}$$



$$|j_1, \ldots, j_N\rangle$$

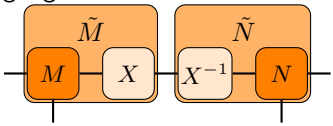
free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?



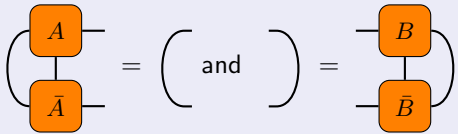
$\to$ in *mixed* canonical form

gauge freedom on contracted legs:

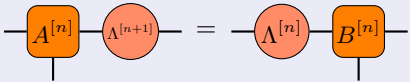


Canonical Form

ONB of Schmidt states



in canonical form with

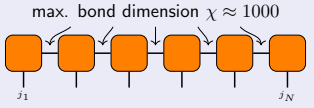


Matrix Product States (MPS)

Definition (MPS)

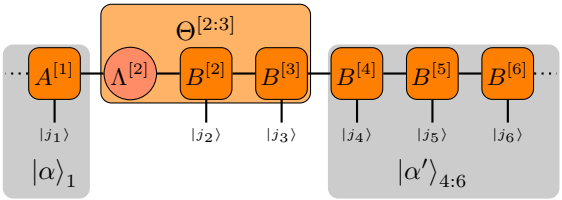
Variational Ansatz

$$|\psi\rangle = \sum_{j_1 \dots j_N} \; |j_1, \dots, j_N\rangle$$



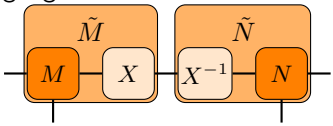
free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?



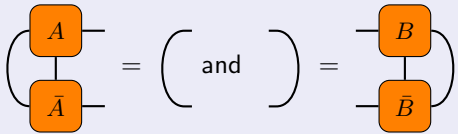
$\to$ in *mixed* canonical form

gauge freedom on contracted legs:

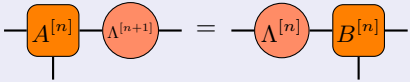


Canonical Form

ONB of Schmidt states



in canonical form with

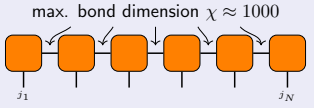


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

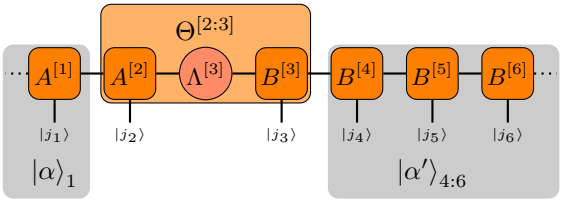
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



$$|j_1, \dots, j_N\rangle$$

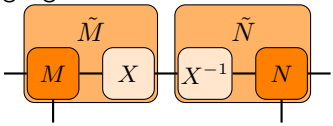
free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?



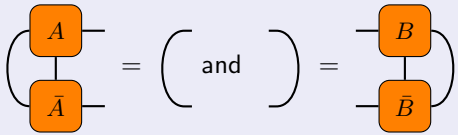
$\to$ in *mixed* canonical form

gauge freedom on contracted legs:

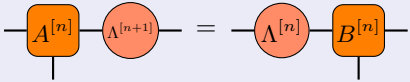


Canonical Form

ONB of Schmidt states



in canonical form with

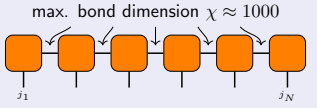


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

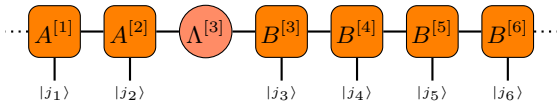
$$|\psi\rangle = \sum_{j_1 \cdots j_N}$$



$$|j_1, \ldots, j_N\rangle$$

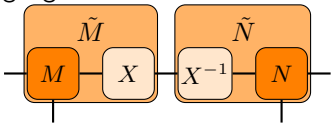
free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?



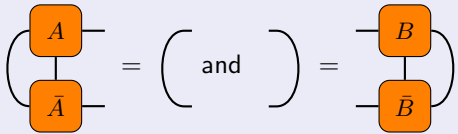
$\to$ in *mixed* canonical form

gauge freedom on contracted legs:

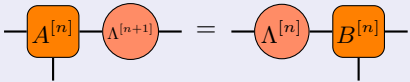


Canonical Form

ONB of Schmidt states



in canonical form with

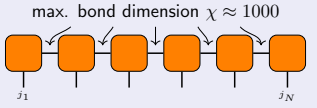


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

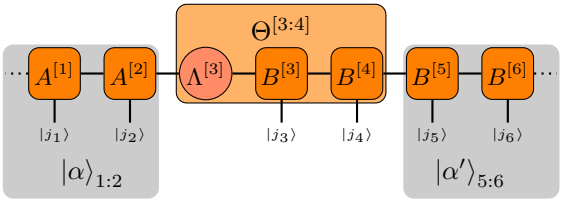
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



$$|j_1, \dots, j_N\rangle$$

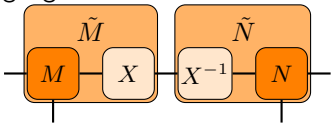
free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

How to initialize given $|\psi\rangle$?



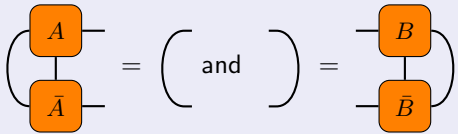
$\to$ in *mixed* canonical form

gauge freedom on contracted legs:

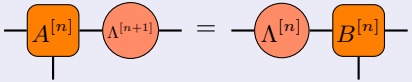


Canonical Form

ONB of Schmidt states



in canonical form with

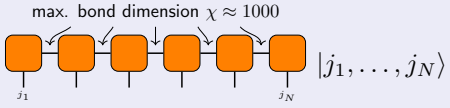


Matrix Product States (MPS)

Definition (MPS)

Variational Ansatz

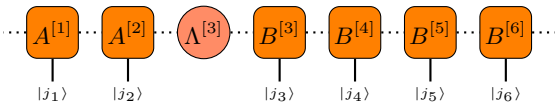
$$|\psi\rangle = \sum_{j_1 \dots j_N}$$



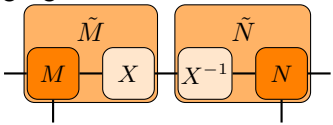
$$|j_1, \dots, j_N\rangle$$

free parameters $\mathcal{O}(d^N) \to \mathcal{O}(N\chi^2)$

in praxis: initialize product state $|\psi\rangle$
with trivial $\Lambda^{[n]} = (1)$

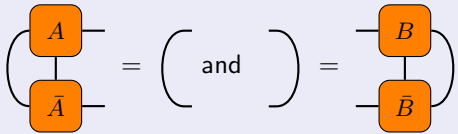


gauge freedom on contracted legs:

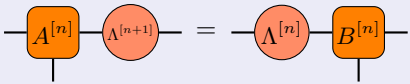


Canonical Form

ONB of Schmidt states

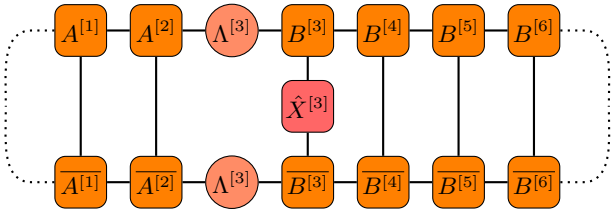


in canonical form with



MPS Expectation values

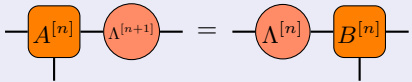
$\langle\psi|\hat{X}_3|\psi\rangle =$



$\Rightarrow$ always contract *locally*, i.e. sideways rather than top to bottom!

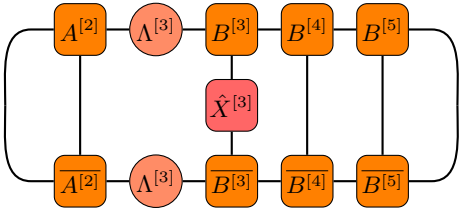
Technical Trick

store $\Lambda^{[n]}$ and $B^{[n]}$ in MPS class
and convert to/from $A^{[n]}$ via



MPS Expectation values

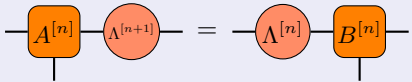
$\langle\psi|\hat{X}_3|\psi\rangle =$



$\Rightarrow$ always contract *locally*, i.e. sideways rather than top to bottom!

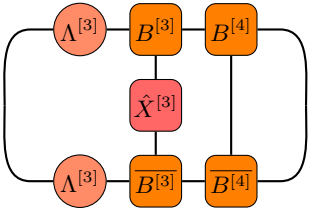
Technical Trick

store $\Lambda^{[n]}$ and $B^{[n]}$ in MPS class
and convert to/from $A^{[n]}$ via



MPS Expectation values

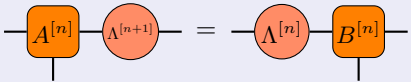
$\langle\psi|\hat{X}_3|\psi\rangle =$



$\Rightarrow$ always contract *locally*, i.e. sideways rather than top to bottom!

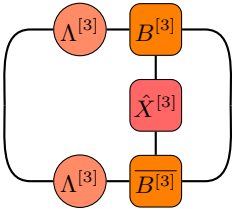
Technical Trick

store $\Lambda^{[n]}$ and $B^{[n]}$ in MPS class
and convert to/from $A^{[n]}$ via



MPS Expectation values

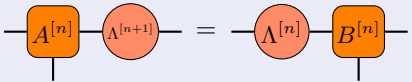
$\langle\psi|\hat{X}_3|\psi\rangle =$



$\Rightarrow$ always contract *locally*, i.e. sideways rather than top to bottom!

Technical Trick

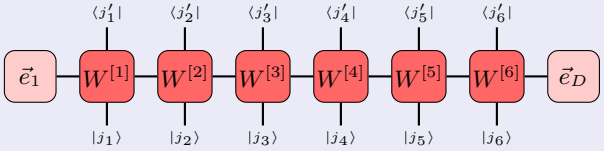
store $\Lambda^{[n]}$ and $B^{[n]}$ in MPS class
and convert to/from $A^{[n]}$ via



Matrix Product Operator (MPO)

Definition (MPO)

write Hamiltonian $\hat{H}$ as



Example (Transverse Field Ising)

$$\hat{H} = -J \sum_i \hat{Z}_i \hat{Z}_{i+1} - g \sum_i \hat{X}_i$$

$$W^{[n]}_{\alpha,\alpha'} = \begin{pmatrix} \mathbb{1} & \hat{Z}_n & -g\hat{X}_n \\ & & J\hat{Z}_n \\ & & \mathbb{1} \end{pmatrix}$$

In general, the MPO form

$$W^{[n]}_{\alpha,\alpha'} = \left(\begin{array}{c|c|c} \mathbb{1} & C & D \\ \hline & \mathsf{A} & B \\ \hline & & \mathbb{1} \end{array}\right)$$

gives rise to

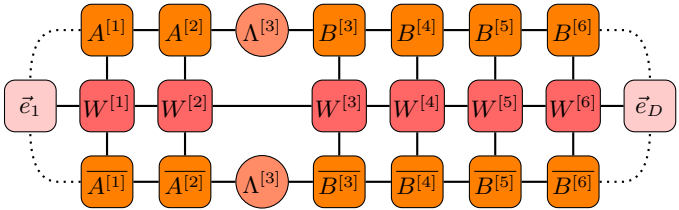
$$\hat{H} = \sum_{i<k} C_i \left(\prod_{j=i+1}^{k-1} A_j \right) B_k + \sum_i D_i$$

$\Rightarrow$ libraries like TeNPy automate MPO construction

$\Rightarrow$ bond dimension D_{MPO} depends on number and range of terms in $\hat{H}$

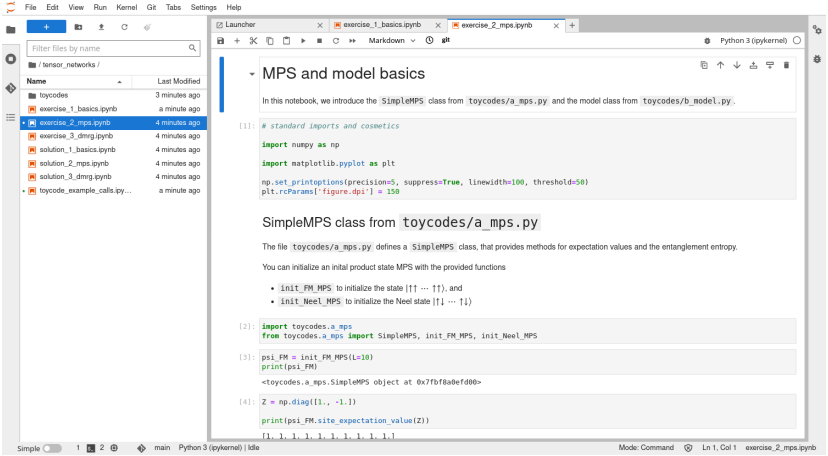
MPO Expectation Value

$$E = \langle\psi|\hat{H}|\psi\rangle =$$



Exercise 2: MPS and Model

https://github.com/tenpy/tenpy_toycodes



Time evolving block decimation (TEBD) algorithm

Suzuki Trotter decomposition

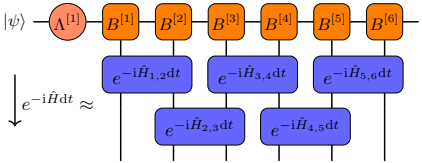
For nearest neighbor model

$$\hat{H} = \sum_n \hat{H}_{n,n+1} = \sum_{n \text{ even}} \hat{H}_{n,n+1} + \sum_{n \text{ odd}} \hat{H}_{n,n+1}$$

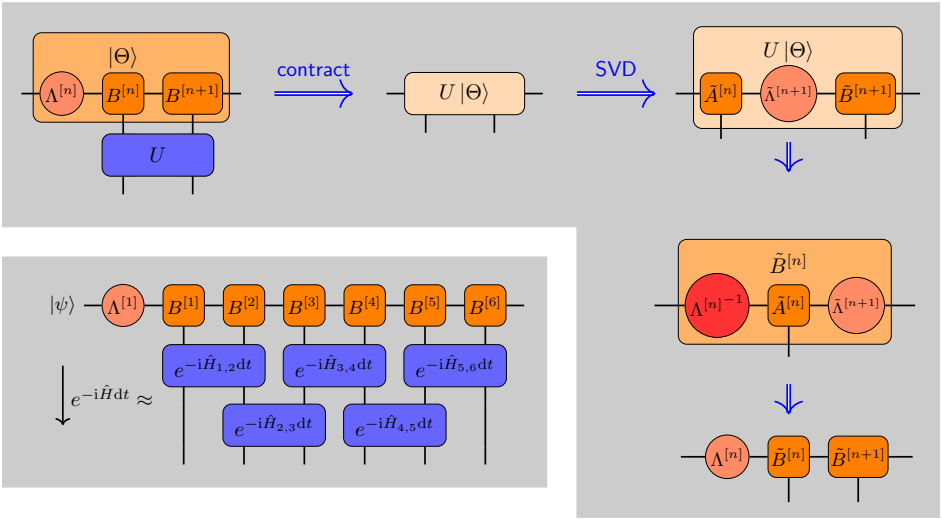
split time evolution $U = e^{-\mathrm{i}\hat{H}t} = \left(e^{-\mathrm{i}\hat{H}\mathrm{d}t}\right)^{N_s}$
into small time steps $\mathrm{d}t = \frac{t}{N_s}$ for which

$$\begin{aligned} e^{-\mathrm{i}\hat{H}\mathrm{d}t} \approx & \quad e^{-\mathrm{i}\hat{H}_{\text{even}}\mathrm{d}t} \quad e^{-\mathrm{i}\hat{H}_{\text{odd}}\mathrm{d}t} \\ = & \prod_{n \text{ even}} e^{-\mathrm{i}\hat{H}_{n,n+1}\mathrm{d}t} \prod_{n \text{ odd}} e^{-\mathrm{i}\hat{H}_{n,n+1}\mathrm{d}t} \end{aligned}$$

up to $\mathcal{O}(\mathrm{d}t^2)$



Time evolving block decimation (TEBD) updates



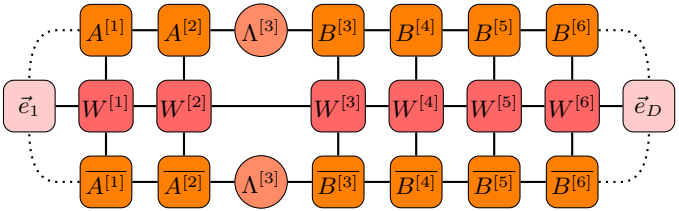
Note: including $\Lambda^{[n]}$ is crucial for optimal truncation!

Density matrix renormalization group (DMRG) algorithm

variationally optimize $|\psi\rangle$
to minimize

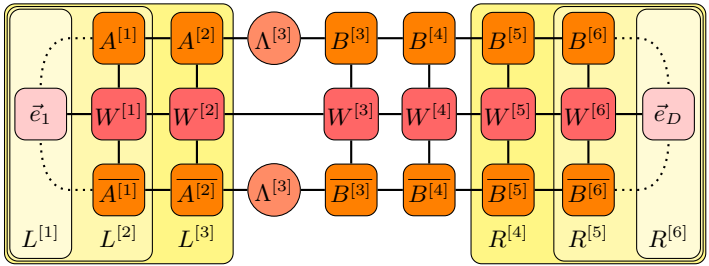
$$E = \langle\psi|\hat{H}|\psi\rangle = \quad \hat{H}$$

$$\langle\psi|$$



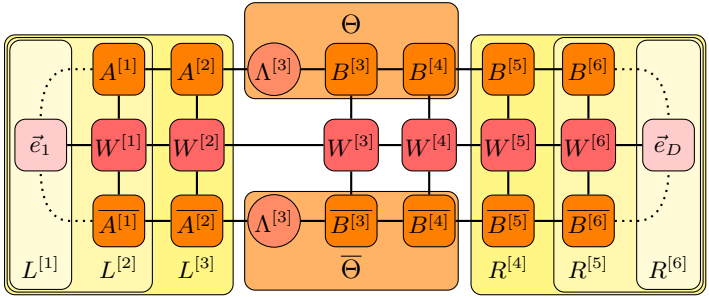
Density matrix renormalization group (DMRG) algorithm

variationally optimize $|\psi\rangle$
to minimize
$E = \langle\psi|\hat{H}|\psi\rangle =$ $\hat{H}$
$\langle\psi|$

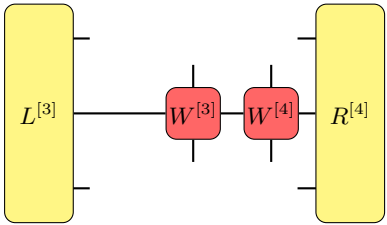


Density matrix renormalization group (DMRG) algorithm

variationally optimize $|\psi\rangle$
to minimize
$E = \langle\Theta|\hat{H}_{\text{eff}}|\Theta\rangle =$ $\hat{H}$
$\langle\psi|$



$\hat{H}_{\text{eff}} =$

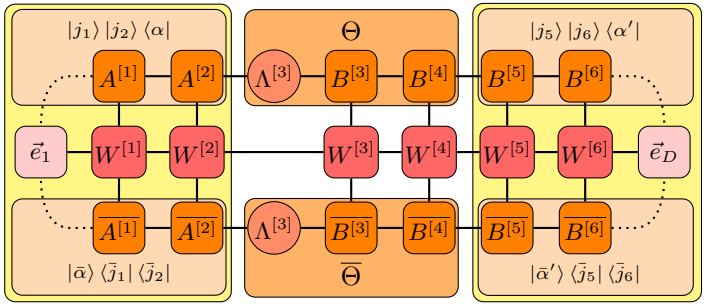


Density matrix renormalization group (DMRG) algorithm

variationally optimize $|\psi\rangle$
to minimize

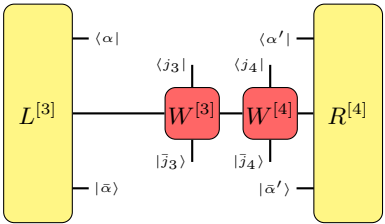
$$E = \langle\Theta|\hat{H}_{\text{eff}}|\Theta\rangle = \quad \hat{H}$$

$$\langle\psi| \quad$$

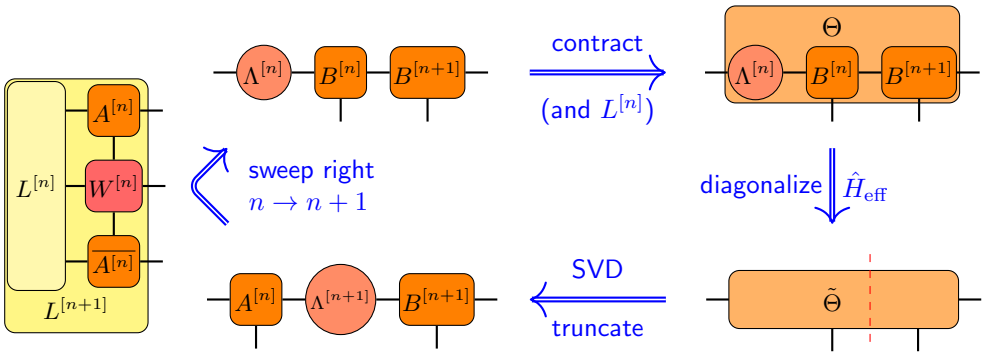


⇒ diagonalize $\hat{H}_{\text{eff}}$
with Lanczos
to update $\Theta \to \tilde{\Theta}$

$\hat{H}_{\text{eff}} =$



DMRG sweep loop

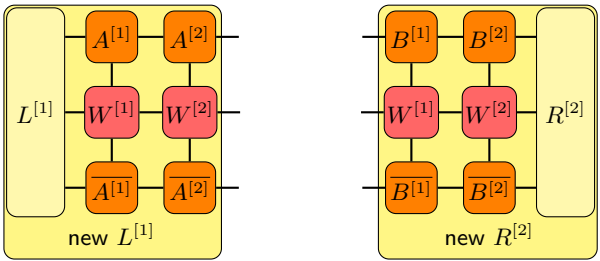


Infinite DMRG

- repeating unit cell of L sites, e.g. $L = 2$,

$$|\psi\rangle = \sum_{\{j_n\}} \cdots B^{[1]}_{j_{-1}} B^{[2]}_{j_0} B^{[1]}_{j_1} B^{[2]}_{j_2} B^{[1]}_{j_3} B^{[2]}_{j_4} \cdots |\ldots, j_{-1}, j_0, j_1, j_2, j_3, j_4, \ldots\rangle$$

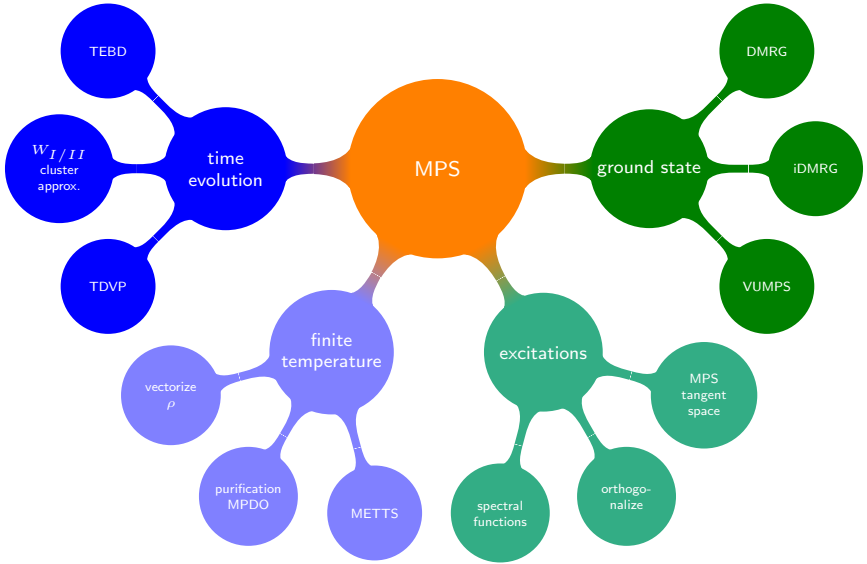
- consider only central unit cell
- grow system: absorb unit cells into environments during sweep



Limitations of MPS

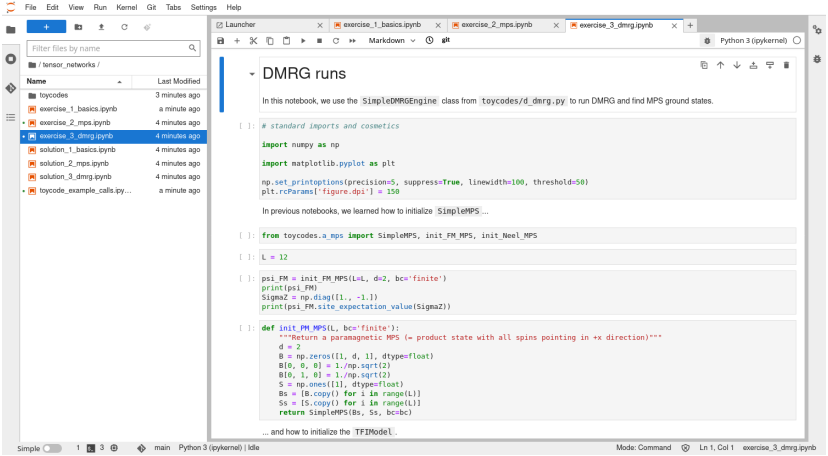
- ✓ memory cost $O(\chi^2 D_{MPO} N)$
- ✓ computation time $O(\chi^3 d^3 N)$ ⇒ $N \gg 1$ or even infinite possible!
- ? Truncation error: $|\psi\rangle$ needs to be representable by MPS
  easy to show that $S \leq \log \chi$ for MPS ⇒ need $\chi \geq e^S$
  - ✓ 1D gapped ground states: area law $S = \text{const}$
  - ? 2D gapped ground states: area law $S \propto L_y$
    - ✓ small $L_y \approx 4$–12
    - ✗ large $L_y \gg 1$ ⇒ fully 2D tensor networks like PEPS
  - ? 1D critical systems: $S = \frac{c}{6} \log L$ ⇒ careful scaling analysis with χ
  - ✓ time evolution of local quench
  - ✗ time evolution of global quench $S(t) \propto t$ ⇒ for short t only!
- ✓ Trotter error from timestep $\mathrm{d}t$ ⇒ smaller $\mathrm{d}t$ and/or higher orders
- ✓ nearest neighbor only for TEBD ⇒ TDVP and $W_{I/II}$ algorithms for general MPO

Outlook: other MPS-based algorithms

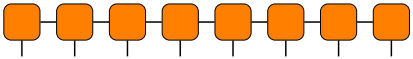


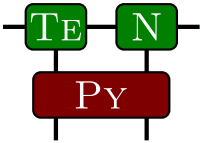
Exercise 3: DMRG

https://github.com/tenpy/tenpy_toycodes



Outline





Intro to MPS, TEBD, DMRG

1. Motivation
2. Exercise 1: Basics
3. MPS
   - MPS Expectation Values
   - MPO for Hamiltonian
4. Exercise 2: MPS and Model
5. TEBD time evolution
6. DMRG ground state search
7. Exercise 3: DMRG

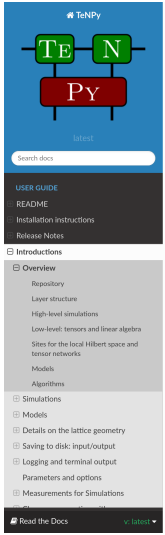
Intro to TeNPy

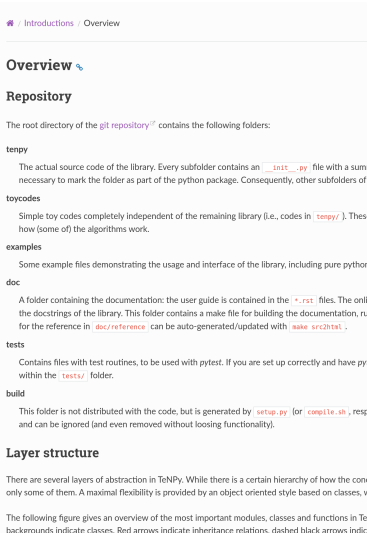
8. Overview
9. Models
10. Simulations
11. Exercise 4: TeNPy

Tensor Network Python (TeNPy)

```
conda install -c conda-forge physics-tenpy
# or alternatively
pip install physics-tenpy
```

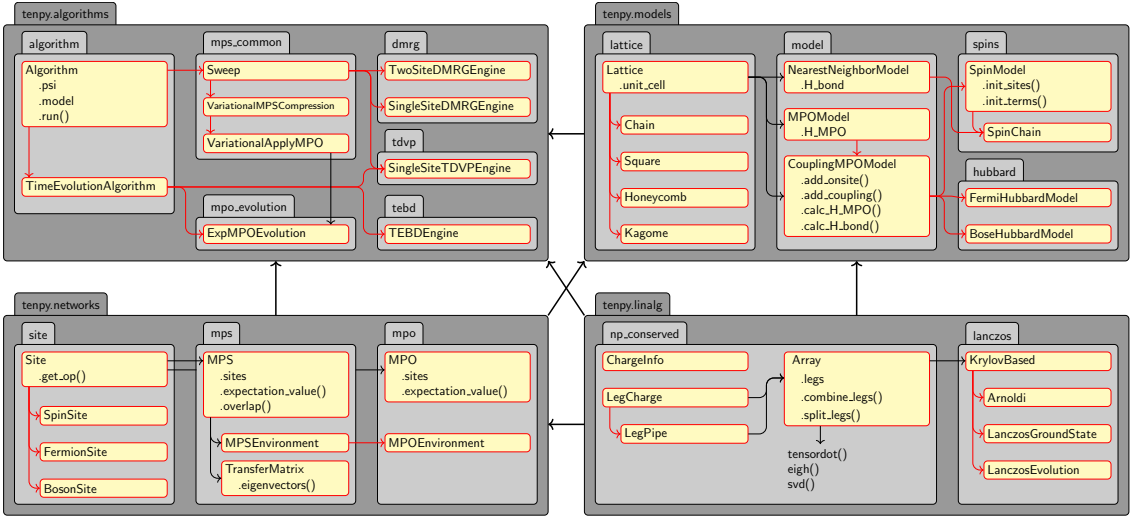
- open source: github.com/tenpy/tenpy
- documentation tenpy.readthedocs.io
- Q&A tenpy.johannes-hauschild.de
- toy codes & examples
- focus: (in)finite MPS in 1D & 2D





Alternatives: ITensor, TensorKit.jl, SymMPS, SyTen, QSpace, TNT, yast, ALPS, ... → tensornetwork.org/software/

TeNPy code overview



Example contractions

```
import tenpy.linalg.np_conserved as npc
from tenpy.networks.site import SpinHalfSite

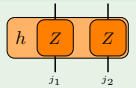
s = SpinHalfSite(conserve='Sz')
Z1 = s.get_op('Sigmaz')
Z1 = Z1.replace_labels(['p', 'p*'],
                       ['j1', 'j1*'])
Z2 = s.get_op('Sigmaz')
Z2 = Z2.replace_labels(['p', 'p*'],
                       ['j2', 'j2*'])

h = npc.outer(Z1, Z2)
h_m = h.combine_legs([['j1', 'j2'],
                      ['j1*', 'j2*']])

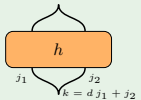
E, V = npc.eigh(h_m)
U = npc.expm(-1.j * 0.1 * h_m)
```

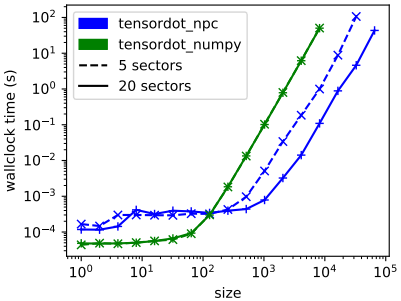
Example (reshape)

$$h := \hat{Z}_1 \hat{Z}_2 =$$

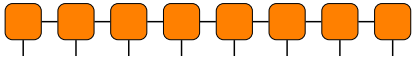


$\simeq$ reshape





Initialize an MPS



```
from tenpy import SpinHalfSite, MPS

s = SpinHalfSite(conserve='Sz')

L = 50
product_state = ["up"] * L
psi = MPS.from_product_state([s] *L,
                             product_state,
                             bc='finite')
print("entropy = ",
      psi.entanglement_entropy())
print("<Z_i> = ",
      psi.expectation_value("Sigmaz"))
print("<Z_i Z_j> = ",
      psi.correlation_function("Sigmaz",
                               "Sigmaz"))
```

Site:

- local basis / Hilbert space
- local operators
- selects charge conservation
  - `SpinHalfSite`
  - `SpinSite`
  - `FermionSite`
  - `BosonSite`
  - `SpinHalfFermionSite`
  - ⋮

Model: represents H

```
from tenpy import SpinHalfSite, CouplingMPOModel

class TFIModel(CouplingMPOModel):
    def init_sites(self, model_params):
        return SpinHalfSite(conserve=None)

    def init_terms(self, model_params):
        J = model_params.get('J', 1.)
        g = model_params.get('g', 1.)
        for u in range(len(self.lat.unit_cell)):
            self.add_onsite(-g, u, 'Sigmax')
        for u1, u2, dx in self.lat.pairs['nearest_neighbors']:
            self.add_coupling(-J, u1, 'Sigmaz', u2, 'Sigmaz', dx)

model = TFIModel(dict(g=1.1,
                      L=50))
```

Example (TF Ising)

$$\hat{H} = -J \sum_{i=1}^{N-1} \hat{Z}_i \hat{Z}_{i+1} - g \sum_{i=1}^{N} \hat{X}_i$$

$$Q = \prod_i \hat{X}_i$$

Model with charge conservation

```
from tenpy import SpinHalfSite, CouplingMPOModel

class TFIModel(CouplingMPOModel):
    def init_sites(self, model_params):
        return SpinHalfSite(conserve=None'parity')

    def init_terms(self, model_params):
        J = model_params.get('J', 1.)
        g = model_params.get('g', 1.)
        for u in range(len(self.lat.unit_cell)):
            self.add_onsite(-g, u, 'Sigmaxz')
        for u1, u2, dx in self.lat.pairs['nearest_neighbors']:
            self.add_coupling(-J, u1, 'Sigmazx', u2, 'Sigmazx', dx)

model = TFIModel(dict(g=1.1,
                      L=50))
```

Example (TF Ising)

$$\hat{H} = -J \sum_{i=1}^{N-1} \hat{X}_i \hat{X}_{i+1} - g \sum_{i=1}^{N} \hat{Z}_i$$

$$Q = \prod_i \hat{Z}_i \simeq \left(\sum_i S_i^z \mod 2 \right)$$

Model with charge conservation in 2D

```python
from tenpy import SpinHalfSite, CouplingMPOModel

class TFIModel(CouplingMPOModel):
    def init_sites(self, model_params):
        return SpinHalfSite(conserve='parity')

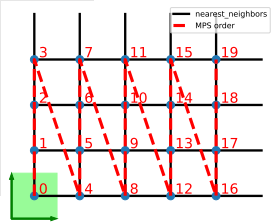
    def init_terms(self, model_params):
        J = model_params.get('J', 1.)
        g = model_params.get('g', 1.)
        for u in range(len(self.lat.unit_cell)):
            self.add_onsite(-g, u, 'Sigmaz')
        for u1, u2, dx in self.lat.pairs['nearest_neighbors']:
            self.add_coupling(-J, u1, 'Sigmax', u2, 'Sigmax', dx)

model = TFIModel(dict(g=1.1, lattice='Square',
                      L=50, Lx=20, Ly=4 })
```

Example (TF Ising)

$$\hat{H} = -J \sum_{\langle i,j \rangle} \hat{X}_i \hat{X}_j - g \sum_i \hat{Z}_i$$

$$Q = \prod_i \hat{Z}_i \simeq \left(\sum_i S_i^z \mod 2 \right)$$



Model with charge conservation in 2D

```python
from tenpy.models.tf_ising import TFIModel

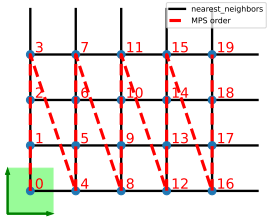
model_params = dict(
    g=1.1,
    conserve='parity',
    lattice='Square',
    Lx=20, Ly=4,
)

model = TFIModel(model_params)
```

Example (TF Ising)

$$\hat{H} = -J \sum_{\langle i,j \rangle} \hat{X}_i \hat{X}_j - g \sum_i \hat{Z}_i$$

$$Q = \prod_i \hat{Z}_i \simeq \left(\sum_i S_i^z \mod 2 \right)$$

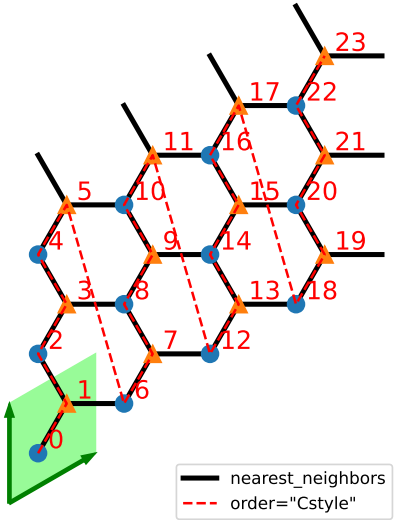


Model Geometry: Lattices

MPS & MPS algorithms are 1D
but the lattice/model can map
2D Hamiltonian $\rightarrow$ 1D with long range

Lattice

- unit cell of a few `Site`
- shift by basis vectors
- ordering how MPS winds
- infinite MPS: repeat in x-direction
- pre-defined lattices
  - `Chain,Ladder`
  - `Square,Honeycomb,Kagome,...`
  - `IrregularLattice`
  - `MultiSpeciesLattice`
  - `HelicalLattice`



A First DMRG Run

```
from tenpy import TFIModel, MPS, TwoSiteDMRGEngine

model_params = dict(L=100, g=1.1, conserve=None)
M = TFIModel(model_params)
product_state = ["up"] * M.lat.N_sites
psi = MPS.from_product_state(M.lat.mps_sites(),
                             product_state,
                             bc=M.lat.bc_MPS)
algorithm_params = dict(trunc_params=dict(chi_max=100))
alg = TwoSiteDMRGEngine(psi, M, algorithm_params)
alg.run() # can run a while...

print("entropy = ",
      psi.entanglement_entropy())
print("<Z_i> = ",
      psi.expectation_value("Sigmaz"))
print("<X_i X_j> = ",
      psi.correlation_function("Sigmax",
                               "Sigmax"))
```

First TEBD Run

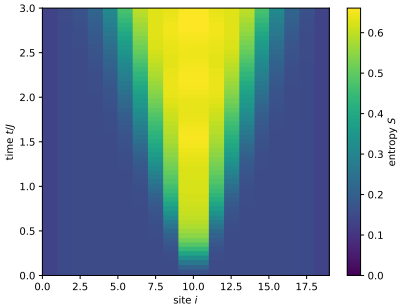
```
from tenpy.algorithms.tebd import TEBDEngine

...

psi.apply_local(psi.L//2, 'Sigmaz')

algorithm_params = dict(
    trunc_params=dict(chi_max=100),
    dt=0.01, order=2, N_steps=5)
alg = TEBDEngine(psi, M, algorithm_params)

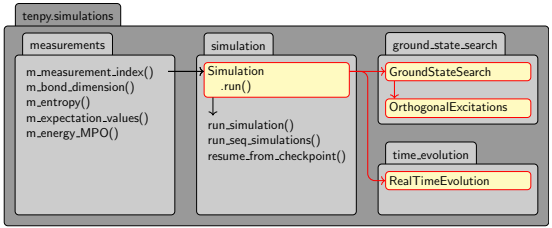
while alg.evolved_time < 3.:
    alg.run()
    print("S =", psi.entanglement_entropy())
    print("X =", psi.expectation_value("Sigmax"))
    print("Z =", psi.expectation_value("Sigmaz"))
```



High-level Simulations

- separate parameters and code
- saves results, logs, checkpoints
- template repo for cluster setup
  github.com/tenpy/project_template
- call as `tenpy-run parameters.yml`

→ `tenpy.run_simulation(**parameters)`



parameters.yml

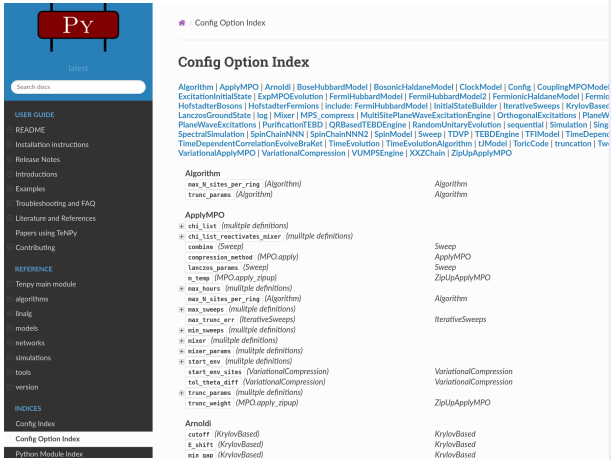
```
simulation_class : GroundStateSearch
output_filename: DMRG_example.h5

model_class: TFIModel
model_params:
    L : 32
    bc_MPS : finite

initial_state_params:
    method : lat_product_state
    product_state : [[up]]

algorithm_class: TwoSiteDMRGEngine
algorithm_params:
    trunc_params:
        chi_max: 100
```

High-level Simulations



parameters.yml

```yaml
simulation_class : GroundStateSearch
output_filename: DMRG_example.h5

model_class: TFIModel
model_params:
    L : 32
    bc_MPS : finite

initial_state_params:
    method : lat_product_state
    product_state : [[up]]

algorithm_class: TwoSiteDMRGEngine
algorithm_params:
    trunc_params:
        chi_max: 100
```

Simulations: Measurements

```
simulation_class : RealTimeEvolution
algorithm_class: TEBDEngine
algorithm_params:
    trunc_params: chi_max: 100
    dt: 0.01
    N_steps: 5
final_time : 3.

connect_measurements:
  - - tenpy.simulations.measurement
    - m_onsite_expectation_value
    - opname: Sz
  - - psi_method
    - wrap correlation_function
    - results_key: '<Sp_i Sm_j>'
      ops1: Sp
      ops2: Sm
```

your_module.py

```
def m_mag_Z(results,
            psi,
            model,
            simulation):
    Z = psi.expectation_value('Sigmaz')
    results['max_Z'] = Z
```

↓

```
connect_measurements:
  - - your_module
    - m_mag_z
```

Roadmap

2018 v0.3.0	• publish open source: DMRG, TEBD
2024, v1.0	• VUMPS • quasi-particle excitations • SpectralSimulation
WIP towards v2.0	• use GPUs • non-Abelian symmetries
wishlist sometime v2.X	• isoTNS • PEPS, MERA • open systems

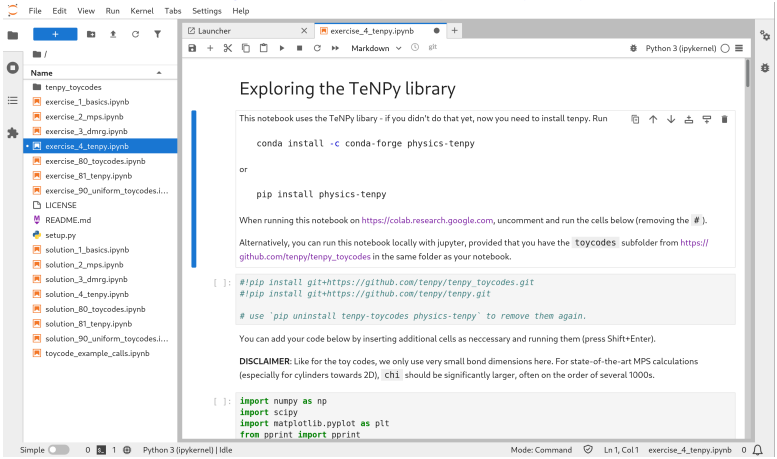
Contributors

Jakob Unfried
Nico Kirchner
Ludwig Zweng
Sajant Anand
Wilhelm Kadow
Kévin Hémery
Philipp Thoma
Jan Geiger
Bartholomew Andrews
Frank Pollmann
Michael P. Zaletel
Roger Mong
Maximilian Schulz
Leon Schoonderwoerd
Gunnar Moeller
Samuel Scalet
Markus Drescher
Stefan Divic
Daniel Parker
Jack Kemp
Michael Rader
Umberto Borla
Yantao Wu
. . . and many more!



Exercise 4: TeNPy

https://github.com/tenpy/tenpy_toycodes



(Some) References

- Yearly schools: European tensor network `quantumtensor.eu`


White, Phys Rev L **69** 2863 (1992)
"Density matrix formulation for quantum renormalization groups"


JH, Pollmann, SciPost Phys. Lect. Notes 5 (2018), arXiv:1805.00055
"Efficient numerical simulations with Tensor Networks: Tensor Network Python (TeNPy)"

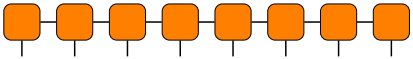

Schollwoeck, Annals of Physics 326, 96 (2011), arXiv:1008.3477
"The density-matrix renormalization group in the age of matrix product states"

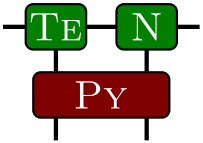

Paeckel *et al.*, Annals of Physics 411, 167998 (2019), arXiv:1901.05824
"Time-evolution methods for matrix-product states"


Vanderstraten, Haegemann, Verstraete, SciPost Phys. Lect. Notes 7 (2019), arXiv:1810.07006
"Tangent-space methods for uniform matrix product states"

see also `https://tenpy.readthedocs.io/en/latest/literature.html`

Outline





Intro to MPS, TEBD, DMRG

1. Motivation
2. Exercise 1: Basics
3. MPS
   - MPS Expectation Values
   - MPO for Hamiltonian
4. Exercise 2: MPS and Model
5. TEBD time evolution
6. DMRG ground state search
7. Exercise 3: DMRG

Intro to TeNPy

8. Overview
9. Models
10. Simulations
11. Exercise 4: TeNPy

Charges: Spin-1/2 with S^z conservation

Consider two spin-1/2 sites with states and

$$H = \vec{S}_1 \cdot \vec{S}_2 = \frac{1}{2}\left(\hat{S}_0^+ \hat{S}_1^- + \hat{S}_0^- \hat{S}_1^+\right) + \hat{S}_0^z \hat{S}_1^z = \sum_{ab} H_{ab} \left|a\right\rangle \left\langle b\right|$$

$$\text{with } H_{ab} = \frac{1}{4}\left(\begin{array}{c|cc|c} 1 & & & \\ \hline & -1 & 1 & \\ & 1 & -1 & \\ \hline & & & 1 \end{array}\right) \qquad \rightarrow \text{block strucuture!}$$

$$\left\{\left|a\right\rangle\right\} \equiv \left\{\left|\uparrow\uparrow\right\rangle, \left|\uparrow\downarrow\right\rangle, \left|\downarrow\uparrow\right\rangle, \left|\downarrow\downarrow\right\rangle\right\}$$

Blocks are identified by charges, here

$$q = (\#_\uparrow - \#_\downarrow) = 2S^z/\hbar \in \mathbb{Z}$$
$$q_{\uparrow\uparrow} = 2, \qquad q_{\uparrow\downarrow} = q_{\downarrow\uparrow} = 0, \qquad q_{\downarrow\downarrow} = -2$$

Charge rule

Define:

LegCharge for each leg $[i]$, with

charges $q_{a_i}^{[i]}$ array of charge values for each index a_i

qconj $\zeta^{[i]}$ "inward" ($+1$, ket) or "outward" (-1, bra).

qtotal Q total charge of a tensor ($\approx$ offset from diagonal)

Charge rule

$$\forall a_1, a_2 \cdots a_r :$$
$$\zeta^{[1]} q_{a_1}^{[1]} + \zeta^{[2]} q_{a_2}^{[2]} + \zeta^{[3]} q_{a_3}^{[3]} + \cdots + \zeta^{[r]} q_{a_r}^{[r]} \neq Q \qquad \text{(charge rule)}$$
$$\Rightarrow \; M_{a_1 a_2 \cdots a_r} = 0$$

(generalization: can have $\mathbb{Z}_n$ instead of $U(1)$,
and more than one charge, e.g., $\mathbb{Z}_2 \times \mathbb{Z}_2$.)

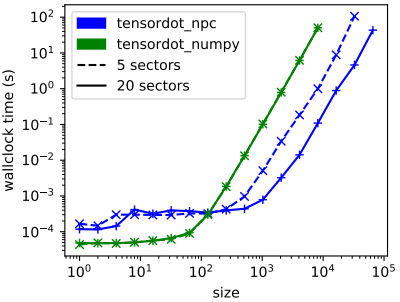
Charges: the gain

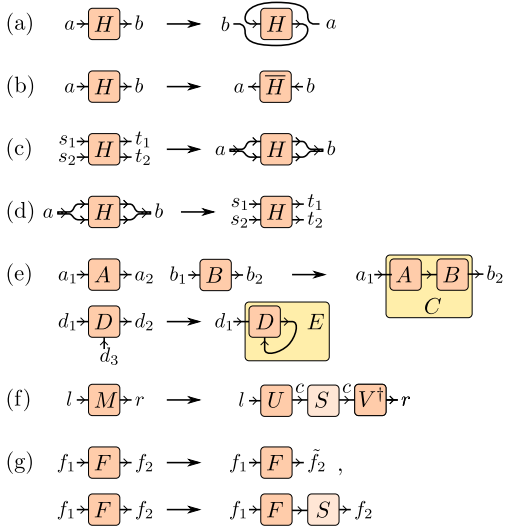
- ensure (charge rule) in `__init__`
- basic operations preserve (charge rule)

⇒ `Array` obey (charge rule) at all times

⇒ exploit (charge rule):

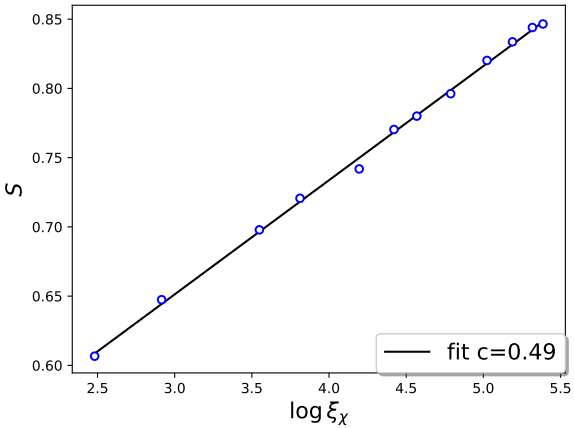
  - store only non-zero blocks
  - operate on blocks





Infinite MPS: Scaling with χ

$$S = \frac{c}{6} \log(\xi) + \text{const.}$$



See `examples/central_charge_ising.py`